DESIGN AND ANALYSIS OF ALGORITHMS

FRACTIONAL KNAPSACK PROBLEM

Greedy Algorithm using Value/Weight Ratio

Learning Outcomes

By the end of this lecture, students will understand:

01
What the Fractional Knapsack Problem is

02
How it differs from 0/1 Knapsack

03
Why value/weight ratio is the correct greedy choice

04
How to solve a problem step by step

05
Why greedy works for fractional knapsack

06
Time and space complexity

Lecture Roadmap

Introduction → Greedy Strategy → Worked Example → Proof & Analysis → Practice

Real-Life Story: The Thief Analogy

A thief has a bag that can carry **limited weight**.

There are many items in a shop. Each item has:

Weight and Value

Goal: Carry the **maximum possible value** without crossing the weight limit.

This is the essence of the Knapsack Problem a classic optimization problem in computer science.

Think about it: If you could take fractions of items (like gold dust or oil), how would you decide what to take first?

Knapsack Family: Two Major Versions

FRACTIONAL KNAPSACK

Items can be broken into parts

Examples: gold dust, oil, grain, liquid

Greedy algorithm works!

You can take 60% of an oil barrel if that fills your bag optimally.

VS

0/1 KNAPSACK

Items cannot be broken

Examples: laptop, painting, phone

Greedy does not always work

You cannot take half a laptop it is either taken fully or left behind.

Formal Problem Definition

GIVEN

n items, each with:

Weight of item i = $\mathbf{w_i}$

Value of item i = $\mathbf{v_i}$

Knapsack capacity = **W**

Fraction taken of item i = $\mathbf{x_i}$

where $\mathbf{0 <= x_i <= 1}$

$x_i = 1$ full item taken

$0 < x_i < 1$ fraction taken

OBJECTIVE

Maximize $\sum x_i v_i$

CONSTRAINT

(\sum x_i w_i \leq W)

Key Idea: Because fractions are allowed, greedy becomes possible. We can always take the "best part" of any item.

Item	Weight (kg)	Value ($)	Ratio
Gold	30	120	4.0

What Should We Choose First?

1 Highest Value

Choose the item with the highest total value first

Is this optimal?

2 Lightest Item

Choose the item with the lowest weight first

Is this optimal?

3 Value/Weight Ratio

Choose the item with the highest value per kg

This is the answer!

? **Question for students:** Which strategy gives the best value per kg of capacity?

Intuitively, we want to maximize the value we get from **each unit of weight** we carry. This leads us to the greedy criterion:

ratio = value / weight

Wrong Strategy 1: Sort by Value

Capacity = 10 kg

Item	Weight	Value	Ratio
A	10	100	10
B	5	80	16
C	5	60	12

✗ Greedy (by value)

Pick A (highest value). Total = **100**, weight = 10 kg. Bag is full. Cannot take B or C.

✓ Optimal

Take B + C. Total = **140**, weight = 10 kg. Same capacity, better value!

⚠ **Message:** Highest value may waste capacity. A heavy high-value item can block multiple better items.

One heavy item (100) vs. Two better items (140) the winner is clear!

Wrong Strategy 2: Sort by Weight

Capacity = 10 kg

Item	Weight	Value	Ratio
A	1	1	1
B	10	200	20

✗ Greedy (by weight)

Pick A (lightest). Total = **1**, weight = 1 kg. Then B doesn't fully fit. Terrible result!

✓ Optimal

Take B directly. Total = **200**, weight = 10 kg. Much better value!

Message: Light item is not always valuable. Weight alone does not indicate worth.

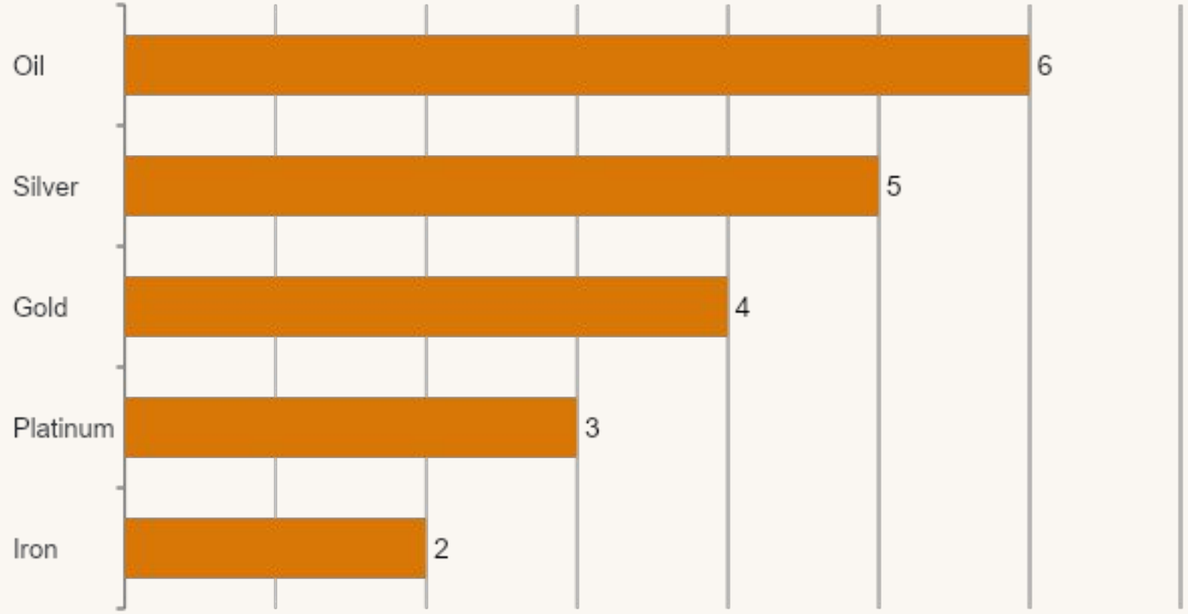
Tiny cheap item (value 1) vs. Heavy valuable item (value 200) weight alone is misleading!

Correct Greedy Criterion

ratio = value / weight

Sort items by v_i / w_i in descending order

This tells us **how much value we get from each kg** of an item. The higher the ratio, the more "valuable per unit weight" the item is.

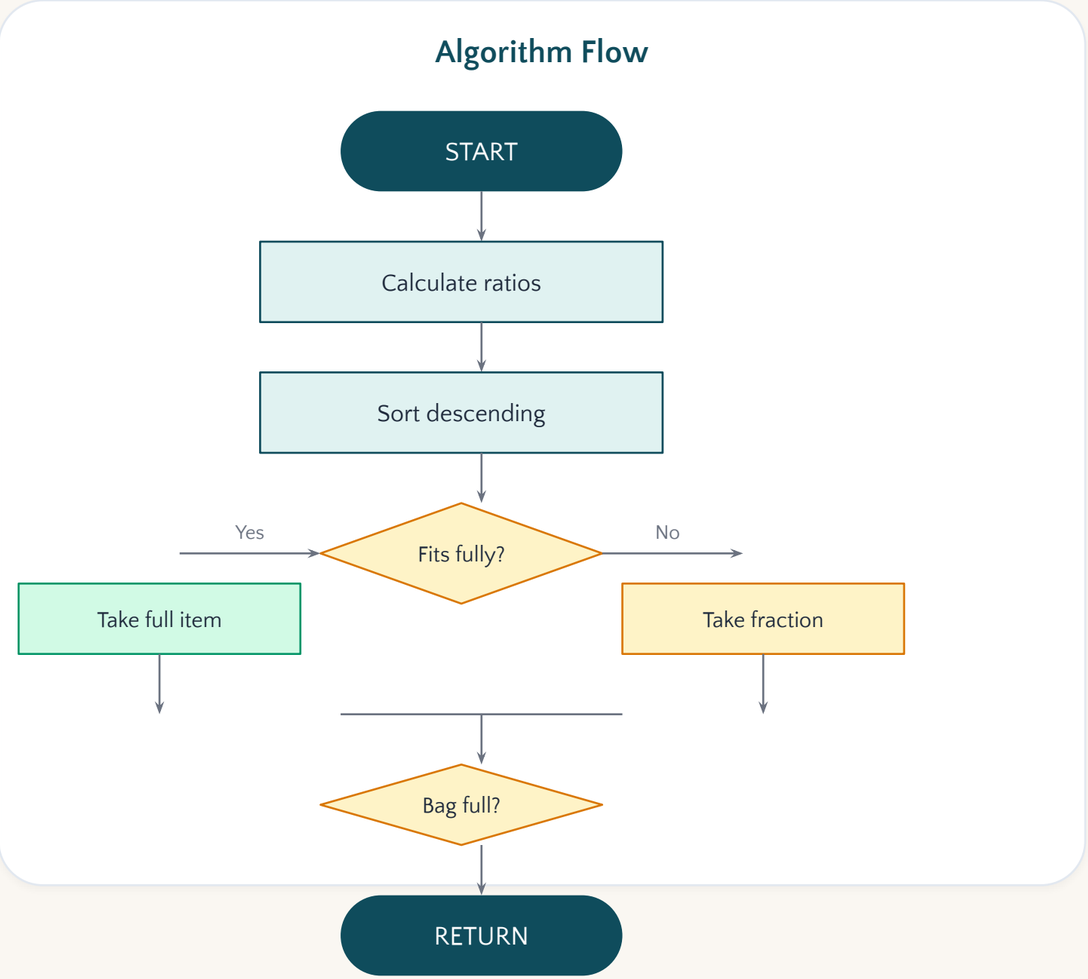


Why This Works

1. We always pick the item that gives the most value per kg first
2. This ensures every kg of capacity is used as efficiently as possible
3. By the time we take a fraction, all better options are already exhausted
4. The result is provably optimal!

Greedy Algorithm Idea

1. Calculate ratio for each item
2. Sort items by ratio descending
3. Start filling the knapsack
4. Take full item if it fits
5. If not fully fit, take required fraction
6. Stop when the bag is full



Pseudocode

FractionalKnapsack.cpp

```
FractionalKnapsack(items, W):
// Step 1: Calculate ratio for each item
for each item: item.ratio = item.v / item.w
// Step 2: Sort by ratio descending
sort(items, by ratio desc)
total = 0; rem = W
// Step 3: Fill greedily
for each item in sorted:
if item.w <= rem:
total += item.v; rem -= item.w
else:
total += item.ratio * rem; break
return total
```

Note: The **else** block handles the fractional case this is the key difference from 0/1 Knapsack!

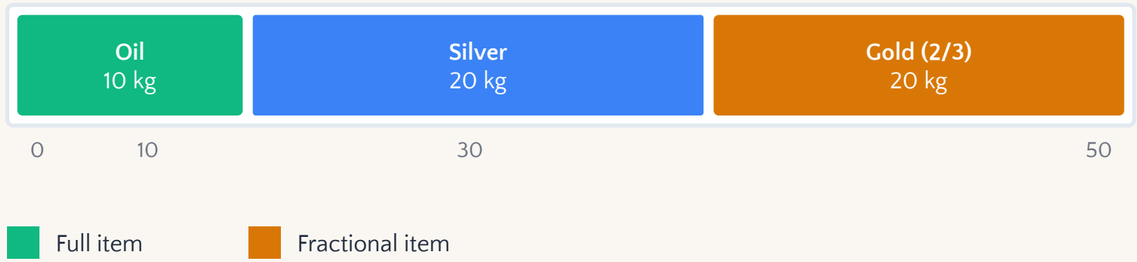
Important Observation

At most ONE item is taken fractionally

WHY?

1. Items before it are fully taken
2. The fractional item fills the remaining capacity
3. After that, the bag is full
4. Remaining items are ignored

Visual: Capacity Bar (50 kg)



Implication: This property is unique to Fractional Knapsack. In 0/1 Knapsack, you might leave unused capacity because the next item doesn't fit there is no "fractional fill."

Worked Example: Setup

W = 50 kg

Goal: Maximize total value

Item	Weight (kg)	Value ($)	Ratio
Oil	10	60	6.0
Silver	20	100	5.0
Gold	30	120	4.0
Platinum	25	75	3.0
Iron	15	30	2.0

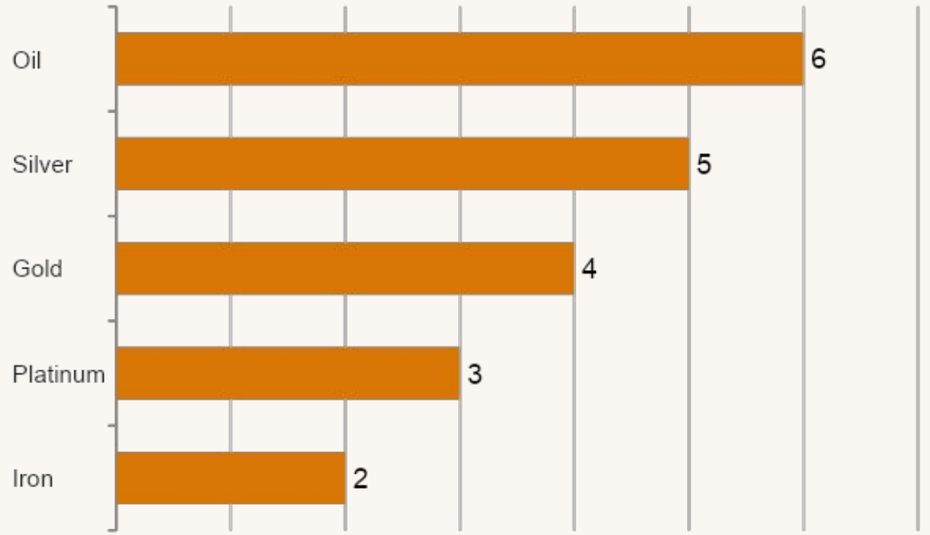
Next, we calculate the value/weight ratio for each item and sort them.

Items Around the Bag

Ratio Calculation

Calculate value / weight for each item

Item	Weight	Value	Ratio	Rank
Oil	10	60	6.0	1st
Silver	20	100	5.0	2nd
Gold	30	120	4.0	3rd
Platinum	25	75	3.0	4th
Iron	15	30	2.0	5th



GREEDY ORDER

Oil → Silver → Gold → Platinum → Iron

Now we fill the knapsack following this order. Let's go step by step!

Step 1: Take Oil

1 of 3

Oil

Capacity = **50 kg**

Oil weight = **10 kg**

Oil value = **$60**

Oil fits fully!

After Taking Oil

Remaining capacity:

50 - 10 = 40 kg

Total value:

$60

Bag Status

Oil 10 kg

Empty: 40 kg remaining

0

50

Next: Silver

Step 2: Take Silver

2 of 3

Silver

Remaining capacity = **40 kg**

Silver weight = **20 kg**

Silver value = **$100**

Silver fits fully!

After Taking Silver

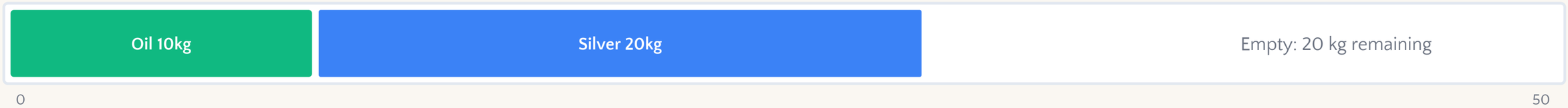
Remaining capacity:

40 − 20 = 20 kg

Total value:

$60 + $100 = $160

Bag Status



Next: Gold →

Step 3: Take Fraction of Gold

3 of 3

Gold

Remaining capacity = **20 kg**

Gold weight = **30 kg**

Gold value = **$120**

Gold does NOT fully fit!

Fraction Calculation

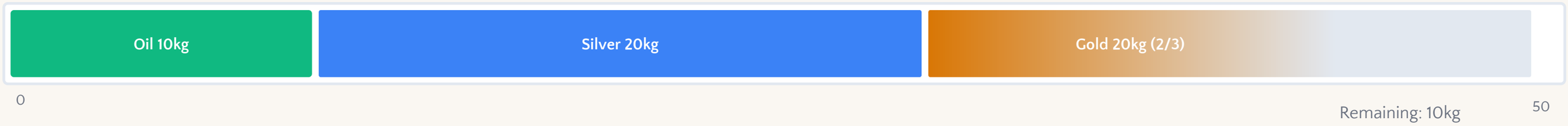
Fraction taken:

20 / 30 = 2/3

Value gained:

2/3 x $120 = $80

Bag Status



Total value so far: **$160 + $80 = $240** | Bag is now FULL!

Final Answer

Maximum Value = $240

SELECTED ITEMS

Oil

Full 10 kg taken, Value = **$60**

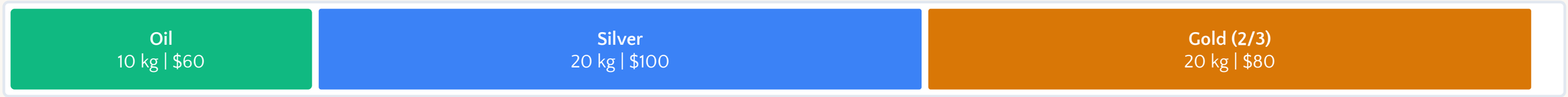
Silver

Full 20 kg taken, Value = **$100**

Gold (2/3)

20 kg of 30 kg taken, Value = **$80**

Final Bag Contents



Total weight: **10 + 20 + 20 = 50 kg** (exactly at capacity)

Verified: This is the optimal solution. No other selection gives more than $240.

Why Greedy Works

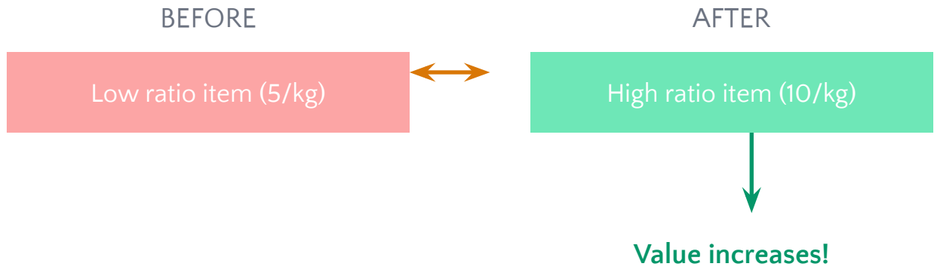
Greedy works because we always take the item with the **highest value per kg**.

If an optimal solution uses a **lower-ratio item** before a **higher-ratio item**, we can **replace** that lower-ratio portion with the higher-ratio portion.

EXCHANGE ARGUMENT

1. Assume an optimal solution O exists
2. If O differs from greedy at any point, it uses a lower-ratio item
3. **Exchange** that portion with the higher-ratio greedy choice
4. Value does not decrease greedy is also optimal

Exchange Argument Diagram



Replacing a lower value/kg item with a higher value/kg item of the same weight always gives **equal or greater** total value.

This proves greedy is optimal.

Key Insight: The exchange argument shows we can always "trade up" to a better solution without losing value.

Exchange Argument in Simple Words

Suppose we have two items:

Item A

10 taka/kg

High value per kg

vs

Item B

5 taka/kg

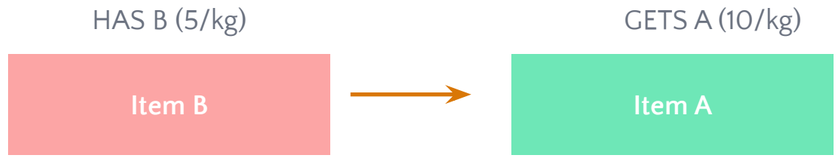
Low value per kg

THE LOGIC

If the bag has some amount of **B** but not enough **A**, then **replacing B with A** gives more or equal value.

So, the greedy choice is **safe**.

Visual: The Swap



Same weight, MORE value!

This is why greedy always picks the highest ratio item first:

1. You can always swap a worse item for a better one
2. The swap never decreases total value
3. Therefore greedy is optimal

The greedy choice is safe we can always exchange low-value-per-kg with high-value-per-kg.

Why This Fails for 0/1 Knapsack

In **0/1 Knapsack**, items **cannot be divided**. So we cannot say "Take only the useful fraction."

Fractional Knapsack

Can take 60% of oil

Can take 1/3 of gold

Greedy works perfectly!

0/1 Knapsack

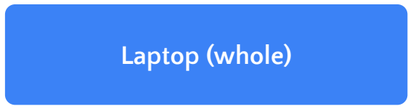
Cannot take half a laptop

Cannot take 1/3 of a painting

Greedy may fail!

The Critical Difference

Fractional: OK



0/1: Cannot split!

An item either **fits fully** or **does not fit**. There is no middle ground.

That is why greedy may fail for 0/1 Knapsack you cannot "fill the gap" with a fraction.

0/1 Knapsack Counterexample

Capacity = 50 kg

Item	Weight	Value	Ratio
A	10	60	6
B	20	100	5
C	30	120	4

✗ Greedy (by ratio)

Picks A + B = value **$160**, weight 30 kg

Remaining = 20 kg, so C **cannot fit**

Suboptimal!

✓ Optimal (using DP)

Takes B + C = value **$220**, weight 50 kg

Uses full capacity perfectly

Better solution!

⚠ **Message:** Greedy fails for 0/1 Knapsack. The ability to take fractions is what makes greedy optimal.

Fractional vs 0/1 Knapsack

Feature	Fractional	
Can split item?	Yes	No
Greedy works?	Yes	No
Main method	Greedy Algorithm	Dynamic Programming
Time complexity	O(n log n)	O(nW)
Example items	Oil, grain, gold dust	Laptop, book, phone

Remember: The key difference is the ability to split items. This single property determines whether greedy is optimal or not.

C++ Implementation Idea

knapsack.cpp

```cpp
struct Item { double w, v, ratio; };

double fractionalKnapsack(vector<Item>& items, double W) {
for (auto& item : items)
item.ratio = item.v / item.w;
sort(items.begin(), items.end(),
[](Item a, Item b) { return a.ratio > b.ratio; });
double total = 0;
for (auto& item : items) {
if (item.w <= W) { W -= item.w; total += item.v; }
else { total += item.ratio * W; break; }
}
return total;
}
```

Key line: totalValue += item.ratio * W handles the fractional case in one elegant line!

Time Complexity

Ratio Calculation

Loop through all n items

O(n)

Sorting

Sort n items by ratio

O(n log n)

Greedy Selection

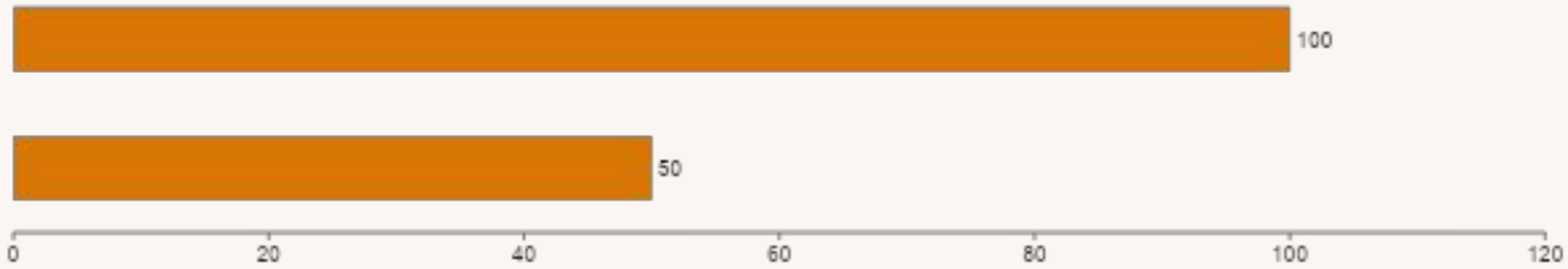
Single pass through items

O(n)

TOTAL

$$O(n) + O(n \log n) + O(n) = O(n \log n)$$

Sorting **dominates** the algorithm. This is very efficient!



Space Complexity

Space Complexity = **O(n)**

We store all items in an **array or vector**. No DP table is needed.

Fractional Knapsack

Space: **O(n)**

Just an array of items

No extra DP table needed

0/1 Knapsack (DP)

Space: **O(nW)**

Requires DP table of size n x W

Much more memory!

Advantage: Fractional Knapsack is much more space-efficient than 0/1 Knapsack!

Common Student Mistakes

Avoid these mistakes!

1. Sorting by **value only**
2. Sorting by **weight only**
3. **Forgetting** to calculate ratio
4. Taking **full item** when only fraction fits
5. Applying greedy to **0/1 Knapsack**
6. Forgetting **at most one** item is fractional

Correct Approach Checklist

- ✓ Calculate value/weight ratio for ALL items
- ✓ Sort by ratio in DESCENDING order
- ✓ Take full items while they fit
- ✓ Take fraction of next item if needed
- ✓ Remember: at most ONE fractional item
- ✓ Do NOT apply to 0/1 Knapsack!

Class Practice

Solve: Fractional Knapsack

Capacity = 60 kg

Item	Weight	Value	Ratio (?)}
A	10	100	?
B	20	120	?
C	30	180	?
D	40	160	?

TASKS

1. Calculate the value/weight ratio for each item
2. Sort items by ratio descending
3. Fill the knapsack step by step
4. Find the maximum value

HINTS

Ratio = Value / Weight

A: 100/10 = **10**

B: 120/20 = **6**

C: 180/30 = **6**

D: 160/40 = **4**

Order: **A, B, C, D**

Try filling the bag!

A (10kg) + B (20kg) = 30kg

C has 30kg left...

Answer: $400

Key Takeaways

1. Fractional Knapsack allows **partial items**
2. Correct greedy criterion is **value/weight ratio**
3. Sort by ratio **descending**
4. Take full items first, then **one fractional item**
5. Greedy is optimal because **fractions allow exchange**
6. Time complexity is **O(n log n)**

7. 0/1 Knapsack is **different** and needs **Dynamic Programming**!

Summary Mind Map

FRACTIONAL KNAPSACK = GREEDY WORKS

Because fractions allow us to always choose the best value per unit weight.

NEXT TOPIC

Job Sequencing with Deadlines